

[Gemini](#)

GC-IMS 數據分析 Python 工具包

<https://share.gemini.google/1k2y7eFJrnsh>

透過「**Flash**」建立 2026 年 8 月 10 日 下午 01:47 發布時間：2026 年 8 月 12 日
上午 08:05

gc-ims-tools 是由 Joscha Christmann、Sascha Rohn 與 Philipp Weller 等學者於 2022 年發表在 Food Chemistry 期刊的開放原始碼 Python 工具包，主要用於 GC-IMS（氣相層析-離子遷移譜）數據的自動化預處理、特徵提取與化學計量學（Chemometrics）建模。

主要核心功能

1. 數據導入與標準化（Data Import & Preprocessing）

- 支持常見 GC-IMS 數據格式的解析與矩陣化（如處理 3D 數據：保留時間 t_R 、漂移時間 t_D 與信號強度）。
- 提供 RIP（反應離子峰，Reaction Ion Peak）位置自動對齊與漂移時間校正（Drift time alignment）。
- 基線平滑與雜訊過濾處理（Baseline correction & Smoothing）。

2. 特徵工程與降維（Feature Extraction & Dimensionality Reduction）

- 自動或半自動的 Peak Finding（峰值識別）與 ROI（Region of Interest）感興趣區域選取。
- 支持主成分分析（PCA），方便進行非受控式的群聚分析與數據可視化。

3. 化學計量學與機器學習建模（Chemometric Analysis & Modeling）

- **分類模型**：整合 PLS-DA（偏最小二乘判別分析）、Random Forest、SVM 等，適用於產地鑑別、摻偽檢測與等級分類。
- **迴歸模型**：支持 PLS 迴歸，用於定量分析與品質指標預測。
- **交叉驗證**：內建交叉驗證與模型性能評估矩陣（ROC、Confusion Matrix）。

4. 高品質圖表輸出（Visualization）

- 可生成標準的 GC-IMS 2D/3D 熱圖（Topographic plots）、比對圖（Difference plots）以及載荷圖（Loadings plots）。

安裝與基礎用法

該套件已上架至 PyPI，可以直接透過 pip 安裝：

Bash

```
pip install gc-ims-tools
```

基本代碼範例

Python

```
import gc_ims_tools as gcit
```

1. 載入 GC-IMS 數據集

```
data = gcit.Dataset.from_folder('path/to/measurement_data')
```

2. 數據預處理：對齊 RIP 與基線校正

```
data.align_rip()
```

```
data.cut_retention_time(start=100, stop=1000)
```

3. 主成分分析 (PCA) 降維與繪圖

```
pca = gcit.PCA(n_components=2)
```

```
pca.fit(data)
```

```
pca.plot_scores(hue=data.labels)
```

4. 建立 PLS-DA 分類模型

```
plsda = gcit.PLSDA(n_components=3)
```

```
plsda.fit(data.X, data.labels)
```

應用價值

- **替代昂貴/封閉的商業軟體**：以往 GC-IMS 數據分析多依賴機器原廠軟體（如 Laboratory Analytical Viewer, LAV），gc-ims-tools 提供免費、開放且可批次處理的 Python 解法。
- **無縫接軌 Python 生態系**：數據結構可直接轉換為 pandas.DataFrame 或 numpy.ndarray，方便結合 scikit-learn 或 PyTorch 進行更深入的 AI/ML 特徵萃取與高級分析。

文獻連結

這篇論文的詳細資訊與相關連結如下：

- **論文題目**：gc-ims-tools – A new Python package for chemometric analysis of GC-IMS data
- **發表期刊**：Food Chemistry, Volume 394, 15 November 2022, 133476
- **作者群**：Joscha Christmann, Sascha Rohn, Philipp Weller

相關存取連結

- **期刊正文 (DOI)**：<https://doi.org/10.1016/j.foodchem.2022.133476>
- **PubMed 條目**：<https://pubmed.ncbi.nlm.nih.gov/35717914/>
- **GitHub 程式碼與說明文件 repository**：<https://github.com/Charisma-Mannheim/gc-ims-tools>
- **配套範例數據集 (Data in Brief)**：
<https://doi.org/10.1016/j.dib.2022.108730>

特徵工程與降維（Feature Extraction & Dimensionality Reduction）

- 自動或半自動的 Peak Finding（峰值識別）與 ROI（Region of Interest）感興趣區域選取。更深入解釋說明 如何做自動話

在 gc-ims-tools 中，實現**自動化 Peak Finding（峰值識別）與 ROI（Region of Interest，感興趣區域）選取**的核心邏輯，是將傳統高度依賴人工拉框（Manual annotation）的經驗法則，轉化為基於**圖像處理與信號矩陣分佈**的演算法。

以下是該套件在背景運作時，實現自動化特徵提取的四大核心機制：

1. 數據陣列化與高斯濾波平滑（Data Smoothing & Denoising）

GC-IMS 原始數據本質上是一個雙時間軸的 2D 矩陣（ $t_R \times t_D$ ）或 3D 熱圖。在進行 Peak 尋找前，演算法會先透過 2D 高斯核（Gaussian Filter）或 Savitzky-Golay 濾波進行空間平滑：

- **去除高頻雜訊**：防止儀器檢測器（Faraday Cup）的電路噪訊被誤判為微小 Peak。
- **背景扣除（Background Subtraction）**：自動計算無峰區間的平均基線強度，並從全圖中扣除，使信號峰（Analytes）與基線有明確的分界。

2. 極值搜尋與區域生長法（Local Maxima Detection & Region Growing）

自動化 Peak Finding 主要採用**局部極值定位與動態邊界延伸**：

- **局部極值搜尋（Local Maxima）**：利用滑動視窗（Sliding Window）在 2D 矩陣上掃描，尋找當前視窗內的信號極大值點（ t_R, t_D ）。只有當該點強度高於設定的信號閾值（Signal Threshold，例如背景噪訊的 3 倍或 5 倍 Standard Deviation）時，才會被標記為候選 Peak 中心。
- **分水嶺演算法（Watershed Algorithm） / 區域生長（Region Growing）**：從定位到的 Peak 中心點開始向外擴展，根據強度衰減的梯度（Gradient）自動決定該 Peak 的物理邊界，藉此解決重疊峰（Co-eluting peaks）的分離問題。

3. 自動化 ROI 邊界定義與形狀擬合（ROI Segmentation & Bounding）

標記出 Peak 中心後，套件會自動將其轉化為化學計量學可用的 ROI 特徵矩陣：

- **邊界框判定 (Bounding Box / Ellipse Fitting)**：演算法會在漂移時間 (t_D) 與保留時間 (t_R) 方向上計算信號的半峰全寬 (FWHM)，自動繪製出長方形或橢圓形的 ROI 範圍。
- **單體 (Monomer) 與二聚體 (Dimer) 配對**：GC-IMS 的典型特徵是高濃度化合物會同時出現 Monomer 與 Dimer 兩個峰。演算法會根據相同的保留時間 t_R ，自動將同一 t_R 軸上的多個漂移峰歸類為同一化合物的關聯 ROI。

4. 跨樣本 ROI 網格對齊 (Cross-Sample ROI Alignment)

這是 gc-ims-tools 實現自動化批次處理 (Batch Processing) 最關鍵的一步。不同批次或樣品的 GC-IMS 分析常伴隨輕微的漂移時間或保留時間偏移：

- **參考樣品建立 (Consensus Map)**：演算法會先分析所有樣品的 Peak 分佈，建立一張全樣品的「共識峰圖 (Master Feature Map)」。
- **質心重疊與漂移容差 (Centroid Clustering & Tolerance Window)**：設定 t_R 與 t_D 的容差視窗（例如 ± 0.2 s for t_R , ± 0.05 ms for t_D ），將所有樣品中落入同一視窗內的 ROI 自動歸納為同一個「特徵變數 (Feature Variable)」。
- **特徵矩陣輸出**：最終自動生成一個可直接用於 PCA 或 PLS-DA 的 Dataframe：列 (Rows) 為樣品，行 (Columns) 為各個自動命名好的 ROI（如 ROI_1 (tR: 245s, tD: 7.8ms)），填入的數值則為該 ROI 的積分面積 (Integrated Volume) 或頂峰強度 (Peak Height)。

程式化調用範例

在 Python 中，這個自動化流程通常只需要數行指令：

Python

```
import gc_ims_tools as gcit
```

```
# 1. 載入批次數據
```

```
ds = gcit.Dataset.from_folder("data_directory/")
```

2. 自動尋峰與 ROI 提取 (設定信號噪聲比門檻 SNR 與視窗大小)

```
rois = ds.find_peaks(  
    threshold=0.05,    # 極限信號強度門檻  
    min_distance=(5, 5), # tR 與 tD 軸上區隔兩個 Peak 的最小距離  
    smooth=True        # 是否啟動高斯平滑  
)
```

3. 自動將提取出的 ROI 積分轉為機器學習特徵矩陣 (Samples $\times$ ROI Features)

```
feature_matrix = ds.get_roi_volumes(rois)
```

透過上述機制，gc-ims-tools 將過去人工標記 ROI 需要數小時的繁瑣過程，縮短為數秒鐘即可完成的自動化數據管線 (Pipeline)。

分水嶺演算法 (Watershed Algorithm) / 區域生長 (Region Growing)：從定位到的 Peak 中心點開始向外擴展，根據強度衰減的梯度 (Gradient) 自動決定該 Peak 的物理邊界，藉此解決重疊峰 (Co-eluting peaks) 的分離問題。更深入說明解釋舉例

在 GC-IMS (氣相層析-離子遷移譜) 的 2D 特徵圖中，重疊峰 (Co-eluting peaks，即保留時間 t_R 相近、漂移時間 t_D 也高度重疊的兩個化合物 signal) 是數據處理時最棘手的難題。

如果單純使用固定長方形框 (Bounding Box) 來截取 ROI，重疊峰會被強行包在同一個框內，導致定量不準或特徵洩漏。分水嶺演算法 (Watershed Algorithm) 與 區域生長法 (Region Growing) 就是為了解決「如何將黏在一起的信號峰精準切開」而引進的空間分割機制。

一、物理直觀理解：地形與水流的倒置

想像 GC-IMS 的 2D 熱圖是一張地形等高線圖：

- 信號強度 (Intensity) 最高的 Peak 頂點，就是山峰。

- 信號強度低的背景噪訊區，就是谷底。

1. 分水嶺演算法 (Watershed) 的運作邏輯

分水嶺演算法將這張地形圖倒置 (Invert)，讓山峰變成「盆地/水井」，谷底變成「山脊」：

1. **注水階段**：從每個已經定位出的 Peak 中心（即倒置後的最低點/水井底）開始注入水。
2. **水面上升**：隨著水位抬高，水會向外擴散（區域擴展）。
3. **建立堤防 (Ridge/Watershed Line)**：當來自兩個不同 Peak 的水即將匯合時，演算法會在兩水交界處築起一道「堤防」。這道堤防的軌跡，就是兩個重疊峰的精準物理邊界。

2. 區域生長法 (Region Growing) 的運作邏輯

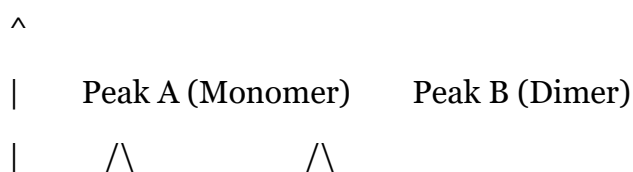
區域生長則是從「種子點 (Seed Point，即 Peak 頂點)」出發：

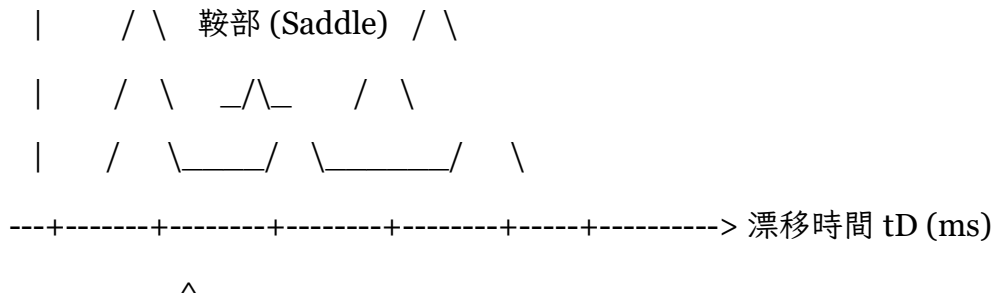
1. 檢查種子點周圍 8 個相鄰像素 (Pixels)。
2. 計算相鄰像素與當前像素的信號強度梯度 (Gradient / $\Delta\text{Intensity}$)。
3. 若梯度變化符合「信號正在向外平滑衰減」的規則，就把該像素納入這個 Peak 的領地；一旦遇到「強度突然止跌回升」（代表進入另一個 Peak 的範圍），生長即刻停止。

二、具體案例說明：分離 Monomer 與 Dimer 的重疊區

以 GC-IMS 中常見的酮類化合物（如 2-Octanone）為例，當濃度較高時，會同時出現單體 (Monomer, M) 與二聚體 (Dimer, D)。假設這兩個峰在 $t_R = 300$ s 處洗脫，且漂移時間 t_D 非常接近（例如 M 在 7.2 ms， D 在 7.6 ms），導致兩峰在底座部分重疊在一起，形成一個「雙峰連綿的山脈」。

信號強度 (Intensity)





分水嶺切割點 (Watershed Boundary)

自動分割步驟拆解：

1. 尋找極值點 (Seeds Placement)

- 演算法透過 2D 矩陣掃描，找到了兩個區域極大值：
 $P_A(300 \text{ s}, 7.2 \text{ ms})$ 與 $P_B(300 \text{ s}, 7.6 \text{ ms})$ 。這兩個點被標記為獨立的「種子」。

2. 計算強度梯度與鞍部定位 (Saddle Point Detection)

- 演算法從 P_A 和 P_B 同時向外生長，並計算沿途的強度變化。
- 在 $t_D = 7.4 \text{ ms}$ 附近，信號強度下降到最低點後開始回升（此處稱為鞍部 **Saddle Point**）。

3. 繪製分水嶺切割線 (Boundary Decision)

- 傳統固定邊界法會畫出一個涵蓋 $7.0 \sim 7.8 \text{ ms}$ 的大長方形，把 A 與 B 混在一起。
- 分水嶺演算法則會沿著鞍部等高線（梯度反轉點）切下一刀，將 3D 空間中的信號雲團精準劃分為：
 - **ROI 1 (Peak A)**：包含 P_A 及其專屬的下坡斜率區域。
 - **ROI 2 (Peak B)**：包含 P_B 及其專屬的下坡斜率區域。

4. 體積積分與定量 (ROI Volume Integration)

- 切割完成後，套件分別對割開後的兩個非規則形狀區域進行信號強度加總 ($\sum \text{Intensity}$)，精準算出 Peak A 與 Peak B 的各自體積 (Volume)，達成高精度的獨立定量。

三、在 gc-ims-tools 中解決 Over-segmentation (過度分割)

分水嶺演算法有一個經典缺點：如果數據中帶有噪訊，噪訊產生的微小波動會被誤判為獨立山峰，導致**過度分割 (Over-segmentation)**——將一個完整的 Peak 切碎成好幾個碎片。

gc-ims-tools 在實作上結合了以下機制來防止此問題：

- **前置 H-minima 變換 / 門檻過濾**：只有當山峰的高度 (Prominence/Depth) 超過設定的信號噪聲比 (如 $\text{SNR} > 3$ 或高於基線一定幅度) 時，才允許作為分水嶺的注水種子。
- **區域合併 (Region Merging)**：在分水嶺切割後，計算相鄰兩個小區域之間的鞍部深度。若鞍部不夠深 (說明只是峰頂的微小噪訊起伏)，演算法會自動將這兩個區域重新合併為同一個 ROI。

你具體案例說明：分離 Monomer 與 Dimer 的重疊區的案例是 RT,跟 DT 都沒有 normalization,實際上是否須要先須要 normalization 處理後再計算或完全不需要，為什麼？

在進行分水嶺演算法 (Watershed Algorithm) 或區域生長法 (Region Growing) 進行重疊峰分割時，「強度/數據的 **Normalization (標準化/歸一化)**」是否需要，完全取決於你處理的是「單一樣品 (**Single Map**)」還是「跨樣品批次 (**Multi-sample Batch**)」。

簡言之：單一樣品內的重疊峰分割「不需要」先做 **Normalization**；但在進行跨樣品的批次分割與特徵比對時，前置的 **Calibration / Alignment** 與 **Normalization** 則是「絕對必要」的。

以下為具體的技術原因與邏輯拆解：

一、為什麼單一樣品內分離 Monomer 與 Dimer 「不需要」 Normalization ？

當我們只聚焦在「單一張 2D 譜圖」上分離 Monomer (M) 與 Dimer (D) 時，分水嶺演算法關注的是矩陣內部的**相對梯度 (Relative Gradient)**與幾何拓撲結構 (**Topology**)，而非數值的絕對大小。

1. 梯度方向 (Gradient Field) 不因線性縮放而改變

- 分水嶺與區域生長法計算的是區域內的**強度變化率** (微分 $\nabla I = \left(\frac{\partial I}{\partial t_R}, \frac{\partial I}{\partial t_D}\right)$)。
- 如果將整張譜圖的信號強度乘以一個常數 k (或做 [0,1] 歸一化)，所有的 Peak 高度與鞍部 (Saddle Point) 強度雖然數值變了，但**極大值點的位置 (Locality of Maxima)**與梯度的正負方向 (**Sign of Derivatives**) 完全不變。
- 因此，分水嶺切出 Monomer 與 Dimer 的「那一刀 (Boundary)」位置是不會改變的。

2. 保留原始光譜/檢測器計數的物理意義

- GC-IMS 的原始信號強度 (如毫伏特 mV 或 Faraday Cup 計數) 具備直接的物理響應意義。若在 Peak 分割前貿然進行非線性的 Normalization，可能會破壞 Peak 的原始形狀 (Lineshape) 與底座衰減比例，反而干擾高斯平滑或梯度判斷。

二、什麼情況下「必須」先做處理？(前置處理的關鍵角色)

雖然不用對信號強度做 [0,1] 歸一化，但在進行重疊峰分割前，有兩種前置處理是決定分水嶺演算法成敗的關鍵：

1. 軸向對齊 (Axis Normalization / Alignment) —— 必須先做

- **漂移時間對齊 (RIP Alignment)**：由於環境溫濕度與氣流微小波動， t_D 軸會有微幅偏移。在分割之前，必須先把 **RIP (反應離子峰)** 的 t_D 位置歸一化到標準值 (如 $t_{D,RIP} = 1.0$ 或固定毫秒數)。
- **為什麼？** 如果 t_D 沒有對齊，設定的極值搜尋視窗 (Sliding Window Size) 或高斯平滑核 (Kernel Size) 就會因為物理尺度的偏差而失準，導致 Monomer 與 Dimer 的搜尋距離條件設定失效。

2. 基線扣除 (Baseline Correction / Background Subtraction) —— 必須先做

- **為什麼？** 分水嶺演算法把低信號區視為「谷底」。如果譜圖存在整體抬升的基線噪訊或 Drift，等同於盆地底部被泥沙填平（鞍部深度被掩蓋），演算法會無法精準找到 Monomer 與 Dimer 之間的鞍部（Saddle Point），導致「過度合併（Under-segmentation）」，把兩峰誤判為同一個大峰。

三、跨樣品 (Batch Processing) 時的 Normalization 時機

如果是要將這套自動化分割套用到 **100 個樣品** 的批次資料庫時：

1. 分割階段 (Segmentation Phase)：

- 各樣品獨立運算時：利用各樣品自身的相對梯度進行分水嶺分割。

2. 定量與機器學習階段 (Modeling Phase)：

- 分割完成並算出 ROI 面積後，必須進行 **Normalization**。
- 因為不同樣品的進樣量、儀器狀態可能微幅漂移，分割出來的 Monomer 與 Dimer 面積 (Integrated Volume) 必須透過 **Internal Standard (內標法)** 或 **Total Peak Area / Vector Normalization (L2 Norm)** 進行標準化，才能送入 PCA 或 PLS-DA 建模。

總結流程順序

在 gc-ims-tools 等現代數據處理管線中，標準的處理順序為：

RIP 對齊 (Axis Alignment) → 基線扣除 (Baseline Correction)
→ 分水嶺重疊峰分割 (Watershed/RegionGrowing)
→ ROI 面積積分 → 強度歸一化 (Intensity Normalization)
→ ML 建模

因此，分水嶺切割本身不需要強度 **Normalization**；它是位在「基線修正後」與「數據歸一化前」的中游關鍵步驟。

[Google 隱私權政策](#)在新視窗中開啟 [Google 服務條款](#)在新視窗中開啟 [個人隱私權與 Gemini 應用程式](#)在新視窗中開啟

Gemini 可能會提供不準確的資訊 (包括人物資訊)，因此請查證回覆內容。

iTo add a conversation: open its : menu → "Move to folder".

?Double-click to rename • Right-click for pin, color, delete.

No folders yet.